MODULE

29

Algorithms & data structures

The classics behind every coding interview, built from scratch and timed so you see Big-O.

YOU WILL LEARN

Sorting & searching	`01_sorting_and_searching.py`
Stacks, queues, linked lists	`02_stacks_queues_linked_lists.py`
Trees	`03_trees.py`
Graphs	`04_graphs.py`
Dynamic programming	`05_dynamic_programming.py`
Problem-solving patterns	`06_problem_solving_patterns.py`
Interview Q&A	`INTERVIEW.md`

WHY CODING INTERVIEWS

Why companies still ask algorithms

THE PROBLEM

In a live coding round you get **45 minutes** and a problem like "find two numbers that add up to a target". The brute force works, but the interviewer asks: **"Can you do better?"**

THE IDEA

Most problems are one of ~8 patterns. Recognise the pattern, and $O(n^2)$ becomes $O(n)$.

STEP BY STEP THE PROCESS

How to solve any coding question

STEP	SAY / DO
1 Clarify	restate the problem; ask about input size, duplicates, empty input
2 Examples	walk through 1-2 small examples by hand
3 Brute force	say the simple solution and its Big-O first
4 Optimise	spot the pattern: hash map? two pointers? sort first? BFS?
5 Code	clean names, small helper functions, talk while you type
6 Test	run your examples + edge cases; state time and space complexity

> Interviewers grade your **communication** as much as the code.

TRY IT SORTING AND SEARCHING

Binary search: halve the problem each step

```
def binary_search(items, target):
    lo, hi, steps = 0, len(items) - 1, 0
    while lo <= hi:
        steps += 1
        mid = (lo + hi) // 2
        if items[mid] == target:
            return mid, steps
        if items[mid] < target:
            lo = mid + 1                    # throw away the left half
        else:
            hi = mid - 1                    # throw away the right half
    return -1, steps

data = list(range(0, 2_000_000, 2))      # 1,000,000 sorted numbers
print(binary_search(data, 1_337_000))
print(binary_search(data, 7))
```

OUTPUT

```
(668500, 16)
(-1, 20)
```

1. Only works on **sorted** data.
2. 1,000,000 items → at most **~20** steps: O(log n).
3. In real code: **bisect**. Python's **sorted()** is Timsort, O(n log n), stable.

STEP BY STEP DATA STRUCTURES

Pick the structure from the question

THE QUESTION SAYS...	USE	WHY
"have I seen it?", counts, pairs	**dict / set**	O(1) lookup
"undo", brackets, "most recent"	**stack** (list)	last in, first out
"in order of arrival", BFS	**queue** (deque)	first in, first out, O(1) both ends
"top k", "smallest first", scheduling	**heap** (heapq)	O(log n) push/pop
sorted data, ranges	**binary search** / BST	O(log n)
connections, networks, grids	**graph** (dict of lists)	BFS / DFS

TRY IT STACKS

Balanced brackets with a stack

```python
def is_balanced(text):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in text:
        if ch in "([{":
            stack.append(ch)                          # push an opener
        elif ch in pairs and (not stack or stack.pop() != pairs[ch]):
            return False                              # wrong closer
    return not stack                                  # nothing left open

for s in ["(a[1] + b) * {c}", "(]", "((x)"]:
    print(f"{s:18} {is_balanced(s)}")
```

OUTPUT

```
(a[1] + b) * {c}   True
(]                 False
((x)               False
```

1. Push openers; every closer must match the **top**.
2. O(n) time, O(n) space.
3. Same idea: HTML tag validation, undo history, call stacks.

PICTURE IT TREES

Binary search trees

🌳 Every node has up to 2 children: **smaller** values go left, **larger** go right. Searching follows one path from the root, so a balanced tree answers in O(log n).

`insert 50, 30, 70, 20, 40`
50 is the root

→

`search 40`
50 → left 30 → right 40

→

`in-order traversal`
left, node, right = SORTED

→

`sorted inserts`
a long chain: O(n)!

Databases use the same idea: **B-tree indexes** (Module 23).

TRY IT GRAPHS

BFS: the shortest path in hops

```
from collections import deque

friends = {"vishal": ["raj", "li"], "raj": ["vishal", "sam"],
           "li": ["vishal", "sam", "priya"], "sam": ["raj", "li", "mia"],
           "priya": ["li"], "mia": ["sam"]}

def shortest_path(graph, start, goal):
    queue, came_from = deque([start]), {start: None}
    while queue:
        node = queue.popleft()
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = came_from[node]
            return path[::-1]
        for nxt in graph[node]:
            if nxt not in came_from:
                came_from[nxt] = node
                queue.append(nxt)
    return []

print(shortest_path(friends, "vishal", "mia"))
```

OUTPUT

```
['vishal', 'raj', 'sam', 'mia']
```

1. BFS explores in **rings**: 1 hop, then 2 hops…
2. Never visit a node twice (**came_from**).
3. Weighted edges (km, ms)? **Dijkstra** with a heap (lesson 04).

TRY IT DYNAMIC PROGRAMMING

Coin change: when greedy fails

```python
def greedy(coins, amount):
    used = []
    for c in sorted(coins, reverse=True):
        while amount >= c:
            amount -= c; used.append(c)
    return used

def min_coins(coins, amount):
    best = [0] + [float("inf")] * amount      # best[a] = fewest coins for a
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                best[a] = min(best[a], best[a - c] + 1)
    return best[amount]

print("greedy:", greedy([1, 3, 4], 6))
print("DP    :", min_coins([1, 3, 4], 6), "coins (3 + 3)")
```

OUTPUT

```
greedy: [4, 1, 1]
DP    : 2 coins (3 + 3)
```

1. DP = solve small subproblems once, store, reuse.
2. Signals: ”fewest / most / number of ways”, overlapping subproblems.
3. Top-down: recursion + **@cache**; bottom-up: a table.

TRY IT PATTERNS

Hash map + sliding window: two classics in O(n)

```python
def two_sum(nums, target):
    seen = {}                                   # value -> index
    for i, n in enumerate(nums):
        if target - n in seen:
            return seen[target - n], i
        seen[n] = i

def longest_unique(s):
    last, start, best = {}, 0, ""
    for i, ch in enumerate(s):
        if last.get(ch, -1) >= start:           # repeat inside the window
            start = last[ch] + 1                # move the left edge
        last[ch] = i
        best = max(best, s[start:i + 1], key=len)
    return best

print(two_sum([2, 7, 11, 15], 9))
print(longest_unique("abcabcbb"), longest_unique("pwwkew"))
```

OUTPUT

```
(0, 1)
abc wke
```

1. **Hash map**: ”have I seen target - n?” in O(1).
2. **Sliding window**: grow the right edge, shrink the left on a repeat.
3. Also: two pointers, prefix sums, BFS/DFS, DP.

COMMON MISTAKE INTERVIEWS

Jumping straight into code

✗ WRONG

```
# reads the question, types for 30 minutes,
# silent, optimises nothing, never tests
def solve(x):
    ...
```

Even a correct answer can fail the round: the interviewer can't follow your thinking and you miss edge cases.

✓ RIGHT

```
# 1. "So the input is ..., output is ..."
# 2. "Brute force is O(n^2) because ..."
# 3. "With a dict I can do it in O(n)"
# 4. code, talking through it
# 5. test: [], [x], duplicates, negatives
```

Think out loud, brute force first, then optimise, then **test**, then state Big-O.

INTERVIEW SAY THIS IN THE INTERVIEW

“Two sum: find indices of two numbers that add up to target.”

- Brute force: check every pair: **O(n²)** time, O(1) space.
- Better: one pass with a **dict** of value → index; for each n, check whether **target - n** is already in it.
- That’s **O(n)** time and O(n) space.
- Edge cases: no solution (return None), duplicates (store after checking), negative numbers (fine).

```python
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return seen[target - n], i
        seen[n] = i
    return None
```

> **Follow-up they ask next:** ”What if the array is sorted?” (two pointers from both ends: O(n) time, O(1) space).

RECAP REMEMBER THESE

Module 29 in 6 lines

1. Process: clarify → examples → brute force → optimise → code → test → Big-O.
2. Binary search: sorted data, O(log n).
3. Stack, queue (deque), heap, dict/set: pick from the question's words.
4. BST in-order = sorted; BFS = shortest path in hops; Dijkstra for weights.
5. DP: overlapping subproblems; greedy can be wrong.
6. Patterns: hash map, two pointers, sliding window, prefix sums.

PRACTICE NOW

Do exercises/ex29_interview_problems.py: 6 classics with a time limit, so brute force won't pass.